41068 ROBOTICS STUDIO 1

Sprint: 4
Version: 1
Date: 30/10/2024

Autonomous Agricultural Monitoring Robot

Crop Guard

24945343	Dylan Purbrick	dylan.m.purbrick-1@student.uts.edu.au
24548238	Alexander Dickson	alexander.s.dickson@student.uts.edu.au
13940085	Andrew Coorey	andrew.g.coorey@student.uts.edu.au
24456156	Anika Roth	anika.roth@student.uts.edu.au
13204591	Jaydie Minter	jaydie.minter@student.uts.edu.au
14273902	Michael Larkham	michael.g.larkham@student.uts.edu.au

Table of Contents

PROJECT DESCRIPTION ***1***

SPRINT 1 ..... ***2***

SLO 1 ..... **2**
SLO 1.1: Communicating with the stakeholder 2
SLO 1.2 Identifying and agreeing on the priorities, goals and system requirements 5

SLO 2 ..... **6**
SLO 2.1 Problem Statement 6
SLO 2.2 Functional requirements and design parameters of the project 6
SLO 2.3 Technical statement 8
SLO 2.4 Design Objectives 9
SLO 2.5 Evaluation criteria 9
SLO 2.6 Timeline for the Sprint 1 10

SLO 3 ..... **11**
SLO 3.1 Configuring the system 11
SLO 3.2 Set up a simple indoor office environment in Gazebo 11
SLO 3.3 Design and develop a subscriber/publisher 12

SLO 4 ..... **13**
SLO 4.1 Time management 13
SLO 4.2 Communication skills 13

SPRINT 2 ..... ***14***

SLO 1 ..... **14**
SLO 1.1 Identifying constraints & risks 14
SLO 1.2 Identify any cultural, ethical, environmental, legislative and economic perspectives 15

SLO 2 ..... **16**
SLO 2.1 Overall Project Idea 16
SLO 2.2 Robot Navigation in Project environment – displaying sensor data 16

SLO 3 ..... **17**
SLO 3.1 Description of a Mapping Task 17
SLO 3.2 Demonstrating Mapping task in Gazebo Environment 17
SLO 3.3 Demonstrate displaying sensors 17
SLO 3.4 Description of a Localisation task 18
SLO 3.5 Localisation task in Gazebo Environment 18
SLO 3.6 Scan Matching localiser 18
SLO 3.7 Compare localisation accuracies against the ground truth 18

SLO 4 ..... **19**
SLO 4.1 Time management 19
SLO 4.2 Effective Collaboration 19

Sprint 3 ..... *20*

SLO 1 ..... **20**
SLO 1.1 Progress update with stakeholder engagement 20

SLO 2 ..... **21**
SLO 2.1 Present the idea of SLAM and path planning 21

SLO 3 ..... **22**
SLO 3.1 Provide a description of SLAM 22
SLO 3.2 Demonstrating the SLAM task in Gazebo environment 23
SLO 3.3 Analyse computational aspects in simulations 24
SLO 3.4 Provide a short description of the Robot Path Planning task 24
SLO 3.5 Demonstrating the path planning task in the Gazebo environment 25

SLO 4 ..... **25**
SLO 4.1 Time management 25
SLO 4.2 Create a GitHub repository 25
SLO 4.3 Professionally formatted portfolio and Doxygen 26

SPRINT 4 ..... *27*

SLO 1 ..... **27**
SLO 1.1 Stakeholder engagement on project completion 27

SLO 2 ..... **28**
SLO 2.1 Develop an evaluation criteria 28

SLO 3 ..... **30**
SLO 3.1 Project evaluation and testing: Overall 30
SLO 3.2 Project evaluation and testing: Mapping 31
SLO 3.3 Project evaluation and testing: Localisation 31
SLO 3.4 Project evaluation: Client's assessment of completeness of the project 32

SLO 4 ..... **32**
SLO 4.1 Time management 32

SLO 5 ..... **33**
SLO 5.1 Critical reflection on the subject learning 33
SLO 5.2 Statement of what you will you do differently 34
SLO 5.3 Peer assessment – Group Assessed "Book Worms" 34
SLO 5.4 Concluding statement 35

References *36*

Appendix *37*

Appendix A: Subscriber & Publisher Flowchart **37**

Appendix B: Risk and Mitigation Strategy Matrix **38**

Appendix C: Overall Project Psuedo Code and High-Level Flow chart **39**

PROJECT DESCRIPTION

The “Crop Guard” project aims to develop an advanced autonomous robotic system designed to enhance agricultural productivity through real-time monitoring of crop and soil conditions. The system will incorporate localization, mapping, and sensor fusion, to navigate crop fields autonomously, detect plant health issues, and provide actionable insights to farmers.

The system will utilise an array of sensors, such as RGB-D cameras, moisture sensors, and nitrate detectors. The system will accurately identify crop discoloration, assess soils quality, and distinguish between crops and weeds. The system’s path planning and obstacle avoidance capabilities will ensure efficient field navigation, even in complex and dynamic environments.

Data collected by Crop Guard is processed in real-time and presented through a user-friendly graphical interface, offering farmers live camera feeds, detailed maps, and easily interpretable reports. The system will also include remote control functionality, allowing manual operation when necessary.

By automating the monitoring process, the system will reduce the need for laborious inspections and external analysis, enabling farmers to make timely, data-driven decisions. This project not only addresses immediate agricultural challenges but also contributes to the broader goal of promoting sustainable and efficient farming practices.

SPRINT 1

SLO 1

SLO 1.1: Communicating with the stakeholder

As part of this project, we are assigned a primary stakeholder who is a tutor, in our case this was Hugh Byun. During this meeting we discussed our project scope and alignment with information from our Real End-user Stakeholder Consultation. Our conversation primarily revolves around the mapping and sensor aspect of this project to ensure we met the assignment requirements.

Due to limited prior knowledge of the tasks required to monitor agricultural crops, we decided to contact a farmer to ask a few questions about monitoring a farm and the current technology available or desired. We were able to get in contact with Andrew who runs a farm in Queensland, and was able to answer all our questions, providing a general direction for the project and key tasks it should complete.

Meeting Name:	Real End-user Stakeholder Consultation	**Meeting Date:**	12/08/2024
Attendees:	Andrew, Alec, Anika, Dylan, Michael, Andrew (farmer)		

Agenda:

- Team to ask questions to end-user about the context of the problem, ideas for solutions and what the end-user needs are.

Summary of Discussion Notes:

- What existing methods are available for agricultural monitoring?
  - Visual inspection
  - Drones
  - Satellite imaging

- What are the limitations that you have faced using these technologies?
  - Visual inspections take a lot of time and can be laborious. This can also be quite dangerous when snakes are about.
  - Drones cannot operate in the rain or when Wi-Fi cuts out.
  - Satellite imaging is expensive, slow and needs to be looked at by a specialist agronomist.

- What types of information would you want to receive from an Autonomous Monitoring Robot?
  - The number and size of weeds in the crop.
  - The colour of the crop (looking for any yellow- or purple-coloured leaves).
  - The soil composition (is it lacking or exceeding in nitrogen, phosphorus, zinc, moisture, acidity, salinity, etc).

- Would you prefer live data, or a collated findings report when the robot operates?
  - Live data is better as the user can see immediately any areas where there are anomalies. Also, if it is scanning a big area, it may take a long time to scan everything and compile a report.

Action items:

- Team will consider the requests and concerns of the end-user and attempt to resolve them in the creation of project solution.

Meeting Name:	Initial Stakeholder Consultation	Meeting Date:	14/08/2024
Attendees:	Andrew, Alec, Anika, Dylan, Michael, Hugh Byun (Client)		

Agenda:

- Team to present project proposal and research collected from real farmer to client. Focus on the scope, problem statement and real farmer's needs.
- Client to clarify the scope of the project and identify areas where project proposal needs adjustment.
- Negotiate terms of consultation (meeting frequency, mode, set up procedure, etc).

Summary of Discussion Notes:

- Choice of sensors:
  - Team suggests using an RGBD camera to inspect crops, radar or lidar to navigate robot (to be finalised) and a moisture and/or nitrate sensor to investigate soil quality (to be finalised). Client approves.
  - Soil quality will be examined by running a randomised script on world initialisation to apply a soil quality value to cells across the map. The robot will then read the data for each cell that it traverses and that will indicate its soil quality. Raw data will be printed to terminal, while processed data will be sent to a GUI.

- Path planning / SLAM:
  - Team proposed having a known environment as the farmer will know how the field is laid out.
  - Client is satisfied with above proposal in this context but recommends that using an unknown environment will show more advanced algorithms for higher marks.
  - Client suggests that team should include obstacle avoidance (e.g. crop has fallen over)
  - Client suggests considering additional functions such as the ability to water plants or trimming tools for pruning.

- Choice of robot platform:
  - Use Turtlebot3 for simulation. Sensors can be added in Rviz and Gazebo.

- Consultation terms:
  - Team can decide consultation frequency. Client suggests a catch up during each sprint.
  - Meetings can be set up through email or by asking for a time to meet in class.

Action items:

- Team to finalise choice of sensors (3 are needed).
- Team to include obstacle avoidance (dynamics or static) in the project scope.
- Team to use Turtlebot3 in simulation.

SLO 1.2 Identifying and agreeing on the priorities, goals and system requirements

List of Priorities

1. Providing a solution which meets the needs of the client/stakeholders
2. Develop an autonomous robotic system capable of navigating a crop field
3. Design the robotic system to be user friendly and safe for farmers to have operating in their fields
4. Meet deadlines specified by client / stakeholders external to project team
5. Complete all assigned sprint work by relevant deadlines
6. Meet all deadlines specified internally by the project team
7. Meet personal deadlines for individual work

List of Goals (in wider context of the project, groupwork, individual work)

- Create a system that offers real-world value to farmers by addressing their key challenges, such as efficient crop monitoring and soil analysis
- Learn and improve proficiency using Linux, ROS, Gazebo, Rviz, OpenCV, and other relevant software
- Successfully simulate a variety of crop conditions in a controlled environment, ensuring the system can accurately detect and report on these conditions.

List of System Requirements:

- The System will run on a Linux operating system using Ubuntu 22.04
- The simulation environment will be set up in Gazebo utilising its physics engine and environment building to replicate real world crop fields
- The System will be developed in a Ros2 workspace and programmed in C++
- The System will use OpenCV for image processing and RVIZ for sensor data visualisation
- The System will require a GUI for the farmer to interact with, providing farmers with live data, control options and notifications of detected issues that require the farmer's attention or action.

SLO 2

SLO 2.1 Problem Statement

Farmers are increasingly seeking innovative solutions to optimise crop monitoring and care throughout the harvesting season. Traditional methods rely on physical inspections and outsourcing crop health analysis to external parties, often leading to delays in identifying issues until significant damage has occurred. These methods also necessitate consultations with third parties to address problem areas in the field, reducing overall efficiency. To enhance productivity, there is a need for an autonomous agricultural monitoring system that can provide real-time data without requiring on-site inspections or external data analysis.

SLO 2.2 Functional requirements and design parameters of the project

The Functional Requirements are formed based on the tasks the stakeholders require or desire the system to perform and the subsequent priorities during these tasks. For example, detecting discolouration is a specific task required by farmers or others who would use the system to monitor agriculture. However, safety considerations such as not damaging the crops was not explicitly stated but would be paramount during the operation to ensure an overall positive contribution to the agricultural practices.

Regarding monitoring discolouration, we decided the system must be able to detect patches of discolouration that indicate typical crop diseases. From scanning the guide for Identification and Management of Field Crop Diseases in Victoria by Grains Research and Development Corporation [GRDC] (2022), we concluded that the indicative colours are dark Orange (rust), yellow, white, black, brown and in some cases purple near the roots.

Through consultations with a farmer, we determined that crop health is primarily monitored through visual inspection. Most farmers would benefit from being able to view the land simultaneously to detect other issues such as water troughs being full or burst pipes, thus we will be incorporating a live video feed. Additionally, checking soil composition, such as nitrate levels and moisture levels of the soil, is important in monitoring crop health; however, it typically requires contracting an agronomist to complete so tests are done sparingly.

Functional Requirements	
The System must detect discolouration on crops (Dark Orange, Yellow, White, Black, Brown, and Purple)	
The System must detect nitrate and moisture levels in the soil	
The System must detect and avoid obstacles in its path	
The System must distinguish between weeds and crops	
The System must return data to the farmer in an easily understandable format	
The System must return a live camera feed to the farmer	
The System must be able to operate completely autonomously	
The System must not damage any of the crops or farmland	
Avg Monitoring Speed	1 crop / 5 sec
The System should be able to autonomously map and navigate a field of crops	
The System should conduct a secondary function to improve crop health	
The System should be able to be remotely controlled by the farmer when desired	

The main physical aspect of this system would ideally be a custom-made robot. Most Design Parameters for the robot come from the fact the robot would be working in a field with farm hands and heavy machinery around, and as such, we decided it would be important for the robot to be easily identifiable amongst the crops. This will also limit any accidents caused by machinery running over the robot, increasing safety and the need for replacements.

Rural areas, especially in Australia, are susceptible to harsh weather patterns, including extended rainfall and/or high temperatures. Therefore, the design of the robot must consider this and be able to continue to function in any weather. The size parameter is influenced by research, which suggests that traditionally, row spacing is 250mm, although this spacing does depend on crop species and farming preferences with, farmers typically preferring larger row spacing (Whitlock, 2013).

Design Parameters	
The Robot must be tall enough to see the tops of the crops	
The Robot must be an easily recognisable colour	
Max Size	250mm X 250mm
The Robot must be waterproof and heat resistant	
The Robot must include easily access to charging ports, USB ports and any ports required for data transfer	
The Robot should emit audible notification of location	
The Robot body should be primarily constructed of easily replaceable materials / parts	

SLO 2.3 Technical statement

Simulation:

Gazebo is to be utilized as the basis of our simulation environment due to the software's robust nature regarding the modelling of environmental scenes and its high-fidelity physics engine. As such, greater accuracy can be achieved in the testing of Crop Guard to ensure that the system is effective, efficient and intelligent. Within the simulation, to emulate the conditions that farmers face, the Gazebo scenario is to launch with a script that changes key parameters of the environment to ensure that the platform would be capable of rolling out to wider markets. Chiefly of these parameters, variable crop conditions will add a level of realism to the simulation allowing for rapid iteration and optimisation of detection algorithms to a robust testing environment.

During the simulation, the Crop Guard robot will manoeuvre autonomously through a simulated field environment, gathering visual and ground probe data on crop health. This data will be stored for a later report and streamed live to the farmer, with visual feed overlays indicating the status of crops as well as the robot's position. This setup not only facilitates real-time monitoring but also enhances the decision-making process by providing actionable insights directly to the farmer. By leveraging Gazebo's comprehensive simulation capabilities, we can refine the Crop Guard system's performance in a controlled environment before deployment in real-world applications.

Choice of Sensors:

The sensor suite equipped onto the Crop Guard robot is vital to interacting with and perceiving both simulated and physical environments. This suite includes radar for navigation in all weather conditions as well as RGB-D, nitrate sensors, and moisture sensors for assessing crop conditions. The information provided by the radar will allow the robot to detect and navigate around obstructions in the path, with the RGB-D cameras able to complete a secondary navigation role while detecting crop health through image recognition. Detection of soil quality will be undertaken by a probe attached to the robot, taking a measurement of water and nitrate content and mapping this to known crop data, such as crop type and age. The use of sensor fusion allows for the robot and farmer to take more directed action, increasing accuracy and certainty of navigation and crop health. The integration of these sensors enhances the system's overall perception capabilities, ensuring better performance in dynamic and varied operational scenarios.

Choice of Platform:

The TurtleBot3 Waffle Pi platform was selected to act as the simulation robot, while a contemporary design will make up the final product for Crop Guard. The TurtleBot platform was desirable for system testing and setting up the simulation due to its ease of use and open-source nature, alongside a wealth of available packages. The Waffle is designed to operate around ROS2, simplifying development. While the custom robot that will become the physical product for Crop Guard will not share the same dimensions and will stand a fair bit taller than the TurtleBot, much of the desired equipment, the overall mobility system and the raspberry pi microcontroller unit. As such the shared components and ease of setting up the TurtleBot within a simulated environment allow for more resources can be focused on improving the system through simulation development, allowing for a faster and more accurate deployment of a physical product.

SLO 2.4 Design Objectives

Required

- Utilises an RGB-D Camera, Radar, moisture sensor and nitrate sensor.
- Uses an RGB camera to detect discolouration, including yellowing, brown (wilting) or white patches on leaves within a 20% error margin.
- Able to detect any weeds by distinguishing between other plants and a known crop that has been planted.
- Can detect and check each crop in a row of 10 crops.
- Live Visual feed from the robot to GUI for farmer.
- Measures soil quality with a probe, noting water and nitrate presence to 2 decimal places within a > 20% false negatives.
- System updates constantly with mapped information available within 10 seconds.
- System does not collide with any environmental objects (crops, structures, etc.) with a full-size simulation.

Desired

- Uploads data at the end of the run with up to 5 days' worth of data held and easily available (based on the farmer's storage).
- Sensor fusion to increase result detection accuracy (should result in > 5% false negatives).
- Can map out and navigate throughout a whole field (simulated 10m x 10m).
- Able to navigate around obstacles blocking the path between crops whilst taking the most direct route.

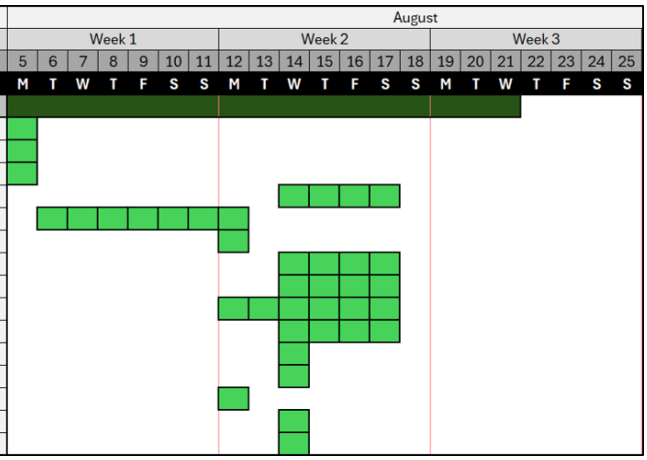
SLO 2.5 Evaluation criteria

Evaluation Criteria	Testing	Done
Incorporates 3 Sensors (RGB-D camera, Radar, Nitrate Sensor)	• Run simulation setup. • Check ROS topics are being published. • Confirm data is being published from desired sensors. • Check system output to confirm data is being received and processed by the system correctly.	☐
Can detect and check each crop in a row of 10 crops	• Run a simulation in an environment with 10 crops in a row. • Check data is being transmitted for 10 crops • Check accuracy of transmitted crop health data	☐
Live Visual feed from the robot to GUI for farmer	• Run a simulation in a test environment. • Check topic lists being published. • Check if there is a live camera feed through RVIZ. • Check if there is a live camera feed on GUI.	☐
Accurate Nitrate Level and Moisture Sensing	• Activate nitrate sensor and moisture sensor. • Run simulation with randomised ground cells. • Info about sensed nitrate and moisture levels is output to the terminal.	☐

	• Compare the data received from simulated nitrate testing to actual values of ground cells. • Compare the data received from simulated moisture detection with actual values of ground cells.	
Identification of Discolouration of crops	• Run the simulation with sections of the crops discoloured (orange, yellow, white, black, brown and pink). • Check log outputs in the terminal to see if the system is recognising the discoloured plants. • Check that the discolouration detected is the accurate colour and associated with the correct plant. • Check GUI to see if the visual representation of discolouration is published.	☐
Object Detection in path	• Run simulation with set obstacles in paths. • System places markers on the detected object. • Terminal output reads object is detected. • Confirm marker location overlaps object. placement in the environment.	☐
Object Avoidance whilst Moving	• Run simulation with set obstacles in paths. • Monitor robot trajectory to confirm path modification for object avoidance. • Confirm the new path completely prevents collision. • Visually confirm if the new path taken by the robot was the most direct around the obstacle.	☐
Collision Avoidance with Environment	• Run simulation with max paddock size and randomization. • Monitor robot movement to ensure it does not collide with any environmental aspects (crops). • Check terminal output to ensure no collisions occurred.	☐

SLO 2.6 Timeline for the Sprint 1

Activity	Assigned To	Status	Start	End
Sprint 1				
Group Formation	All	Complete	5-Aug	5-Aug
Project Selection	All	Complete	5-Aug	5-Aug
Stakeholder Mapping	All	Complete	5-Aug	5-Aug
Team Charter	Michael	Complete	14-Aug	17-Aug
Project Scoping	All	Complete	6-Aug	12-Aug
Problem Statement	Andrew	Complete	12-Aug	12-Aug
Functional Requirements	Anika	Complete	14-Aug	17-Aug
Technical Statement	Alec	Complete	14-Aug	18-Aug
Design Objectives	Anika	Complete	12-Aug	12-Aug
Evaluation Criteria	Dylan	Complete	14-Aug	18-Aug
End-User Meeting	All	Complete	14-Aug	14-Aug
Client Meeting	All	Complete	14-Aug	14-Aug
Create Trello Board	Michael	Complete	12-Aug	12-Aug
Create Gantt Chart	Michael	Complete	14-Aug	14-Aug
Create Gantt Chart	Michael	Complete	14-Aug	14-Aug



SLO 3

SLO 3.1 Configuring the system

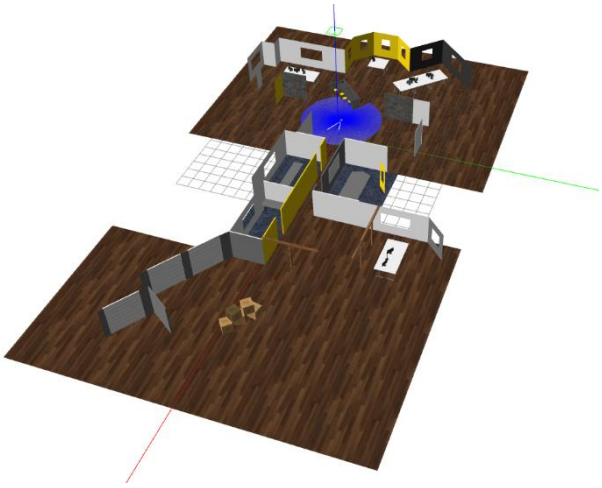
Having already set up the linux environment for PFMS previously, all that I needed to do was to reinstall ROS 2 and RVIZ due to issues with file linkage and pathing. While setting up the system for this subject there were some minor issues when installing the files for ROS which just required me to go back and redownload the required ROS 2 packages.

SLO 3.2 Set up a simple indoor office environment in Gazebo

```
CMake Error at /opt/ros/humble/share/ament_cmake_core/cmake/symlink_install/ament_cmake_symlink_install_targets.cmake:67 (message):
  ament_cmake_symlink_install_targets() 'laser' is not a target
Call Stack (most recent call first):
  /opt/ros/humble/share/ament_cmake_core/cmake/symlink_install/install.cmake:37 (ament_cmake_symlink_install_targets)
  CMakeLists.txt:24 (install)


---
Failed   <<< Wk2_TutrlePerception [1.92s, exited with code 1]
Aborted  <<< cpp_pubsub [1.93s]
Aborted  <<< turtlebot3_description [1.98s]
Aborted  <<< vrc_heightmap_1 [0.36s]
Aborted  <<< dynamixel_sdk [2.10s]
Aborted  <<< dynamixel_sdk_custom_interfaces [2.10s]
Aborted  <<< sprint1 [2.18s]
Aborted  <<< turtlebot3_msgs [2.36s]
Aborted  <<< turtlebot3_gazebo [2.76s]
Aborted  <<< turtlesim [1.78s]
```

Several issues arise while trying to set up the environment, namely issues with gazebo refusing to launch certain environments and getting stuck loading. Ultimately this error was fixed when the old model office_small.world was deleted along with the launch file and downloading the office_cpr.world file and creating the corresponding launch.py file



SLO 3.3 Design and develop a subscriber/publisher

In order to create a subscriber there are 2 key lines, the first is in the constructor when the node is initialised. The this key word is used to associate the subscriber for that specific node to the desired topic. Within the brackets are the parameters to creat the subscription, the first being the name of the topic to be subscribed to, in this case "/scan", next is the buffer which is left as 10 as is typical. The std::bind function is what binds the subscriber to the callback function allowing for the callback function to be run every time the subscriber receives data.

```
scan_sub_ = this->create_subscription<sensor_msgs::msg::LaserScan>("/scan", 10, std::bind(&sprint1::laserCallback, this, std::placeholders::_1));
```

The second line is the initialization of the subscriber shared pointer.

```
rclcpp::Subscription<sensor_msgs::msg::LaserScan>::SharedPtr scan_sub_;
```

Creating a publisher is similar to creating a subscriber in that it involves 2 key lines of code, one for definition and one for initialization. The definition line can be seen below, this also uses the this keyword for the same reason. The key difference is the lack of std::bind function as the publisher, instead the parameters only contain the name for the topic created for the published data and the buffer (once again left as 10).

```
 scan_pub_ = this->create_publisher<sensor_msgs::msg::LaserScan>("/pub", 10);

rclcpp::Publisher<sensor_msgs::msg::LaserScan>::SharedPtr scan_pub_;
```

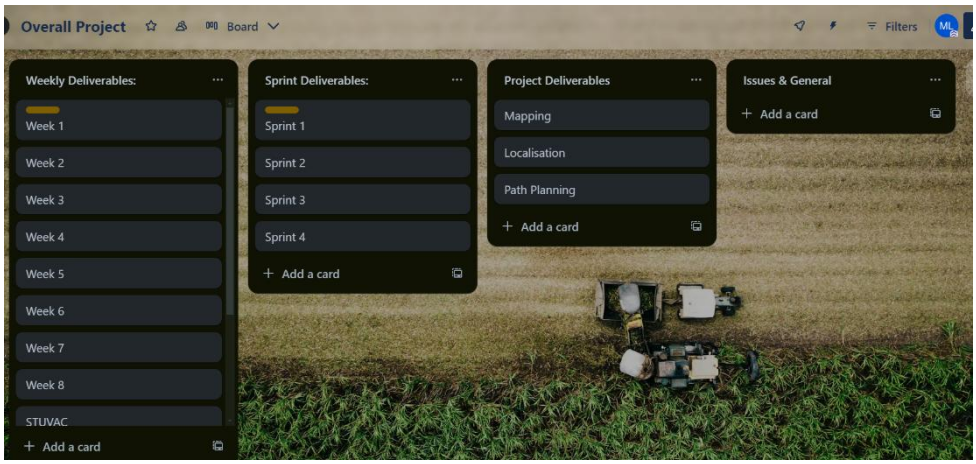
In order to find and publish only the n-th scan points the for loop increases in increments of n. Then to access the necessary information to push into the shared pointer holding the modified scan the algorithm uses i as a substitute index rather than calculating the index based on the angle of the scan point. To successfully publish LaserScan Data it requires the ranges, the min angle, the max angle and the angle increment. The ranges are pushed back into a temporary vector of floats which then fulfils the ranges aspect of the modified scan. The min angle is set when the first interval is found, this is tracked using a boolean. The max angle is constantly updated with each n interval and therefore will be set as the final interval by the end of the for loop. The Angle increment is found by multiplying the original angle increment by n.

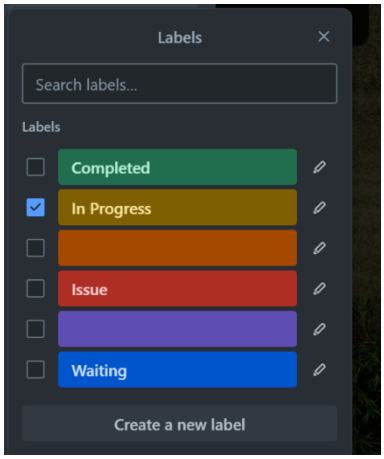
See appendix A for general flowchart of the algorithm.

SLO 4

SLO 4.1 Time management

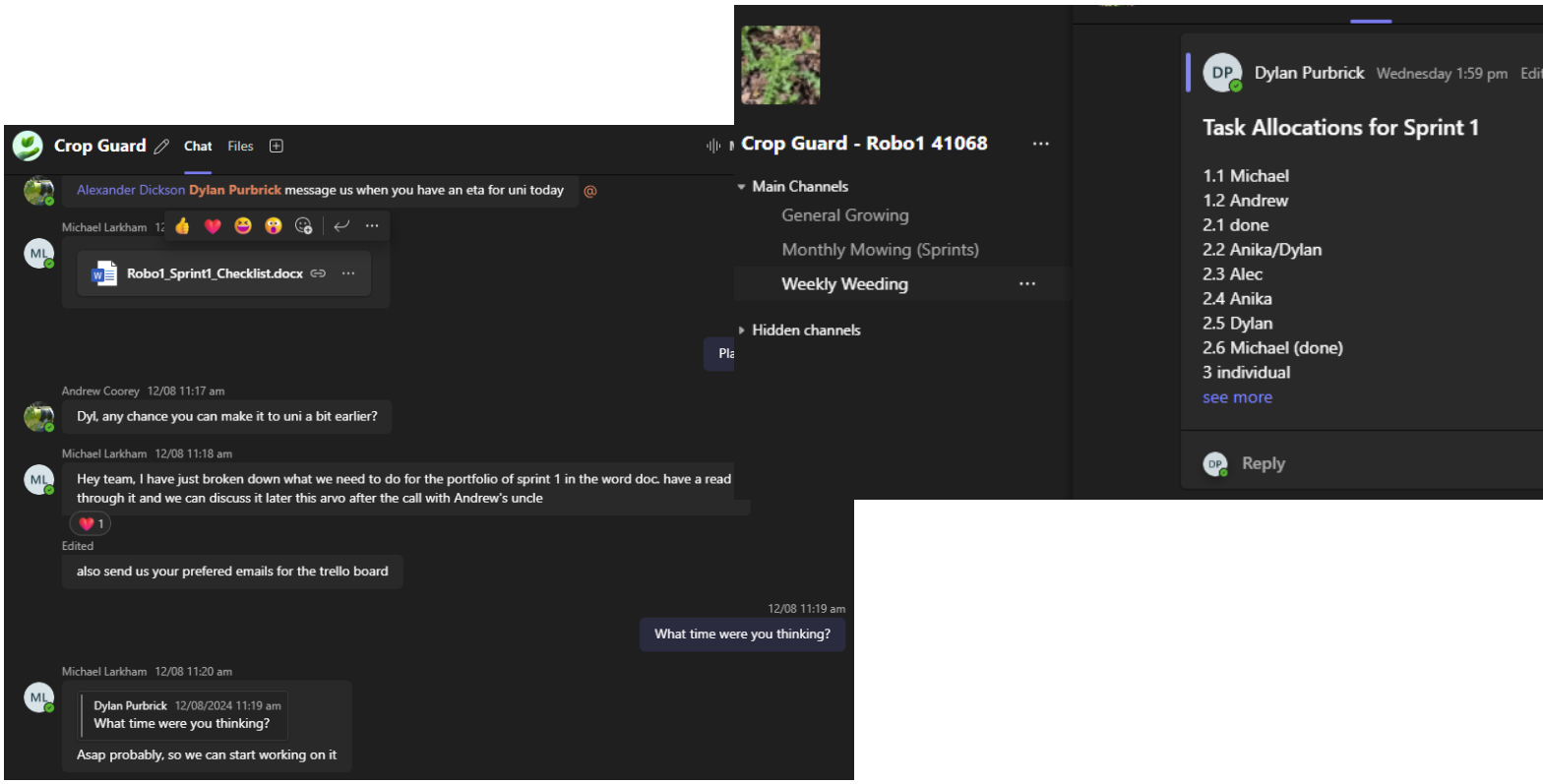
Michael created and shared a Trello Board to better organise the team and ensure all tasks are allocated and due dates are made aware to the team. This board breaks the tasks for the project into 4 tabs, those being weeks, sprints, project deliverables, and issues and & general.





SLO 4.2 Communication skills

The team has chosen to use MS Teams as our main form of communication. Using teams has allowed for easy setting up of meetings and weekly check ins, as well as file storage and document storage.



SPRINT 2

SLO 1

SLO 1.1 Identifying constraints & risks

List of constraints

Hardware:

- Computational resources including laptop hardware such as RAM, processors, graphics chips, etc.
- Sensor resources including their availability, accuracy, and data return information.

Software:

- Using Ubuntu 22.04 operating system considering its limits and capabilities.
- Using ROS2 considering its compatibility and capability.
- Using Gazebo and Rviz considering their compatibility and capability.
- Algorithm complexity to ensure codes can be executed on intended hardware.
- Software integration including connecting independent functional codes and managing organised real-time communications between them.

Simulation:

- Mimicking realism by using realistic health values and conditions.
- Mimicking real farm conditions including, physical layout, terrain, obstacles, weather, etc.
- Mimicking realistic sensor data by ensuring that simulations are not idealistic by adding noise and inconsistencies.

Project:

- Project development time
- Team expertise
- Project budget
- Project scope boundaries (including those imposed by the client)
- Legal obligations including safety requirements, relevant design standards etc.

End-User:

- System must be user friendly and easy to operate for its intended users.
- System must provide useful data and information to allow end users to make useful and informed decisions.

The risks and mitigation strategy matrix can be found ***here*** or in appendix B.

SLO 1.2 Identify any cultural, ethical, environmental, legislative and economic perspectives

Cultural:

- Farmers may be opposed to their monitoring system being fully autonomous and would like a more interactive role in the process, the autonomy would take away from their traditional cultural roles on the farm.
- The autonomous system would improve the working conditions for farmers and farm hands who no longer need to be involved with monitoring the crops, they would be able to focus on their management plan for the crops.
- The autonomous system will reduce the need for farmhands and completely cut out the need for agronomists who the farmers will have formed strong relationships with over the years.

Economic:

- The implementation of the autonomous agricultural system is expected to reduce the costs associated with manual labour and outsourcing crop analysis, thereby increasing the profitability of farming operations.
- By optimising resource usage and improving crop yield, the system contributes to the economic sustainability of farms, ensuring farmers can maximise their returns while minimising input costs.
- The real time data analysed by the system will reduce the need for external consultations and services, allowing the farmers to manage their crops more efficiently and economically.
- Although the initial cost of the autonomous system would be relatively high, the money generated from the improvement in crop yield would be made back over multiple harvest seasons.

Environmental:

- The autonomous agricultural monitoring robot is designed to reduce the environmental impact of traditional farming practices by optimizing resource usage such as water and fertilizers, minimizing the need for harmful chemical applications.
- The system aims to enhance sustainability in agriculture by enabling real-time monitoring of crop health, leading to more efficient and targeted interventions, thereby reducing waste and promoting eco-friendly farming methods.
- By providing early detection of crop diseases and soil deficiencies, the system helps prevent large-scale crop failures, thereby reducing the environmental footprint associated with replanting and over-farming.

SLO 2

SLO 2.1 Overall Project Idea

Within the primary loop, 2 functions will repeat; scanning then navigation but prior to entering a loop, the primary call will use the initialisation function.

Initialisation automatically launches simulation on program launch, starting with the Gazebo environment, then waiting for the user to initialise the ROS nodes.

When scanning, RGB-D camera information is read, processed and then displayed to the GUI. Then the most recent radar reading is taken, and a nested localisation function is called. This function will retrieve the processed camera information and radar reading and dead-reckoning prediction to determine a projected current location that will be used to set a current odometry.

The navigation function will determine if the robot has reached the outer bounds of the field area to end the program, otherwise it will continue to seek out crop conditions along the expected route, marking problematic crops.

The reasoning behind the chosen flow, depicted by the flow chart in Appendix C or here, can be found in the priority of orders, initialisation should only occur once while the first scan must occur before navigation to ensure the robot is localised. Past the first scan, it must still be taken before it's relative navigation command to try to keep as accurate as a localisation prediction as possible.

SLO 2.2 Robot Navigation in Project environment – displaying sensor data

There were several challenges in setting up the agricultural environment for the project. Importing plant models proved difficult, as the available options were not ideal for the crop models. After selecting the best available model, it was imported into a Gazebo world with background models for aesthetics. However, when running RVIS and Teleop commands the plant model showed no laser scan points and the TurtleBot could move through the plant. To fix this, hours were spent delving into the XML file of the world where it was found that there was a collision code written for background tree objects but not for the plant objects. After learning the style of XML code, the collision code for the plants was written and saved into the XML document. This resolved the issues returning laser scan points on the plant models and the TurtleBot could no longer pass through the plants.

The laser scan data was insufficient with each plant returning 3 laser points at maximum which would not be enough information for object detection and navigation. To address this, the scale of the plants was increased in the XML code, resulting in more laser scan points during testing. To complete the environment, a wall resembling a wooden fence was made around the crops. This setup forms the basic environment for the project. Future improvements include creating more suitable plant models and adding plants with discolouration to simulate disease and damage.

SLO 3

SLO 3.1 Description of a Mapping Task

Mapping is the creation of a spatial representation of an environment and is constructed through data gathered from sensors such as LIDAR, RADAR, and RGB-D cameras. LIDAR operates by emitting laser pulses to measure distances based on the time it takes for the light to return after hitting an object, allowing for highly accurate 2D or 3D maps to be created. RADAR uses radio waves to detect objects and measure their distance and speed, performing effectively even in poor conditions like fog or dust. RGB-D cameras capture both colour (RGB) and depth (D) data, enabling the robot to create detailed 3D maps of its environment, which includes both visual and spatial information. The use of this data allows for object detection, obstacle avoidance, and a better understanding of the area being occupied. Mapping data can be represented in both 2D and 3D formats.

Mapping presents several challenges. The most common of these challenges arise from sensor inaccuracy and the noise introduced by the sensors themselves, making it difficult to create an accurate map and complicating navigation. Environmental factors such as lighting, dust, or weather conditions like rain further add to the noise and inaccuracies, reducing sensor reliability. To mitigate this, filters like the Kalman filter are commonly used, particularly in conjunction with LIDAR or other continuous data streams, helping to reduce noise and produce a more accurate map.

SLO 3.2 Demonstrating Mapping task in Gazebo Environment

The environment setup was much the same as SLO 2.2. Minor issues arose due to the size of the world upon the first two attempts of opening the world, but after opening the teams smaller test environment with only 3 rows of crops the larger 5 row environment proceeded to open without issues.

SLO 3.3 Demonstrate displaying sensors

Issues with displaying the odometry and the IMU data within rqt plot initially, but after restarting my device and rerunning the commands I was then able to plot the data without further issues.

SLO 3.4 Description of a Localisation task

Localization is the process of determining a robot's position and orientation within its environment. By using sensor data, such as GPS for outdoor positioning, an IMU to measure acceleration and orientation, and LIDAR for distance measurement and mapping, the robot can estimate its location relative to its surroundings. This enables the robot to navigate effectively and perform tasks autonomously. Localization is critical for ensuring that the robot can move accurately within both known and unknown environments, adapting to changes as needed.

Localization presents several challenges, including sensor drift, where sensors lose accuracy over time, making it difficult for the robot to maintain a precise location. Environmental interference, like GPS signal loss or obstructions, can further complicate the process. Additionally, dynamic environments introduce moving objects, which can confuse sensor readings and hinder accurate localization. Real-time processing in large or complex environments also poses a challenge due to computational limitations.

SLO 3.5 Localisation task in Gazebo Environment

Due to mismanagement of time and troubleshooting in previous tasks this was left incomplete as of submission, but further time will be put towards this until complete.

SLO 3.6 Scan Matching localiser

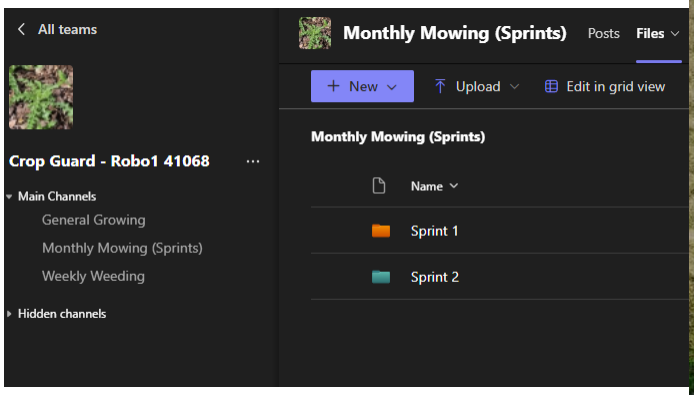
An attempt was made achieve this by the team but due to mismanagement of time we were unable to complete the task. Further work will be put towards this by the team.

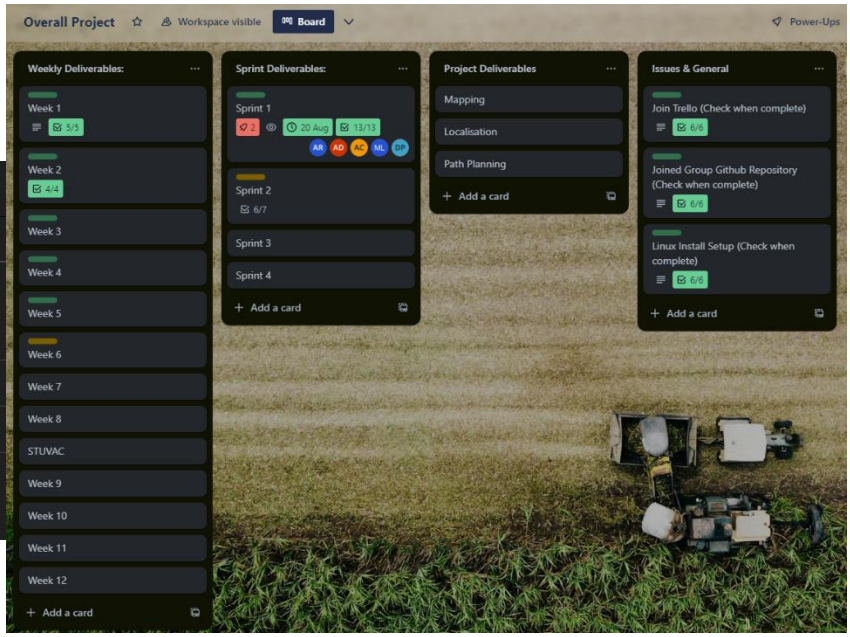
SLO 3.7 Compare localisation accuracies against the ground truth

Due to the incomplete nature of SLO 3.6 SLO 3.7 was unable to be completed. Again, further time and effort will be put towards the completion of SLO 3.7.

SLO 4

SLO 4.1 Time management





Using the Trello board created during sprint 1 by Michael and updated by the team as we work through each week, the team and I have been able to continue working effectively to the deadlines set. Through use of Microsoft Teams, we have been able to continue effective communication and ensure each member is held accountable and made aware of the timeline that we have decided to work to and share files necessary to deliver on sprint 2 deliverables.

SLO 4.2 Effective Collaboration

The team worked together to complete both SLO 3.6 and 3.7 but due to mismanagement of time and complexity of the tasks we were unable to complete what was required.

Sprint 3

SLO 1

SLO 1.1 Progress update with stakeholder engagement

Meeting Name:	Progress Update with Stakeholder	Meeting Date:	25/09/2024
Attendees:	Andrew, Alec, Anika, Dylan, Michael, Hugh Byun (Client)		

Agenda:

- Team to demonstrate project progress including environment, mapping and other developments.
- Client to advise issues with current progress and expectations of future project direction.
- Team to communicate issues being faced and areas of project scope that need adjustment.

Summary of Discussion Notes:

- Current Progress
  - Client is satisfied with the current progress by the team including environment creation and preliminary mapping tasks.

- Future Progress
  - Client and team discussed methods for inclusion of camera and soil quality data. Mutual understanding and agreement were reached on proposed methodology.
  - Team stated intention to use a predefined path and map due to the nature of the project. Client is satisfied with this approach in the project context but advised that team must optimise the path planning algorithm such that it works every time.
  - Client gave team permission for team to use existing Nav2Goal and SLAM packages for the project, but said yaml files must be manually optimised.

- Final submission
  - Team inquired about the nature of the submission being group or individual. Client clarified that final submission is to be individually assessed in a VIVA style assessment, but the code can be identical across all team members. All team members must be fluent in understanding and explaining the code.
  - Team members may use simplified (Azure-friendly) environment for testing but must justify its use in the VIVA if the full environment cannot be demonstrated.

Action items:

- Team may proceed with project as intended continuing to explore alternative mapping and SLAM methods.
- Team to finalise and standardise project environment for Azure and dual-booted operating team members.
- Team to begin examining methods of programming simulated soil quality data array and colour detection.

SLO 2

SLO 2.1 Present the idea of SLAM and path planning

The Crop Guard project will implement SLAM functionality to enhance its crop surveying ability by enabling stronger obstacle avoidance. While using SLAM, the Turtlebot3 will be able to compare its predefined map, against current laser scan data. This will help itself to localise and to distinguish between landmarks and obstacles. Additionally, if the scope of the project were to be broadened to include unique environments, SLAM could have been used to map and localise the robot as it explored new environments. The use of SLAM also provides useful functionality to farmers as they need not waste time scouting their fields and calibrating the robot's location within it.

In the Crop Guard project, integrating effective path planning algorithms will be essential in ensuring the robot's trajectory is both achievable and optimised. Although the robot will be given an ideal path to follow as crops will be fixed, dynamic path planning algorithms will aid in collision avoidance and reducing operational time. For example, if the robot were to encounter a fallen plant blocking its intended path, through using methods such as A* or Rapid-exploring Random Tree (RRT), it will be able to calculate the most efficient path around the blockage to resume its intended trajectory.

SLO 3

SLO 3.1 Provide a description of SLAM

List three sensors that can be used for a robotic SLAM task.
What are the challenges in SLAM? List and briefly describe two popular SLAM techniques in ROS and identify type of sensors suitable for these algorithms.
(Altogether max 300 words)

Simultaneous Localization and Mapping (SLAM) is the process by which a robot constructs a map of an unknown environment while simultaneously determining its own position within that space. This is essential for autonomous robots, as it allows them to navigate and interact with dynamic environments without prior knowledge of the area. SLAM uses various sensors to gather data from the surroundings, enabling the robot to avoid obstacles, detect objects, and localize itself accurately. Its importance lies in the ability to explore, adapt to, and operate within both static and changing environments.

The following sensors are commonly used in SLAM tasks:

1. **LIDAR**: Measures distances by emitting laser pulses and receiving the reflection, generating detailed 2D or 3D maps of the environment.
2. **RGB-D Camera**: Captures both colour and depth data, useful for visual mapping and object detection.
3. **IMU (Inertial Measurement Unit)**: Tracks the robot's orientation and movement, assisting in position estimation.

SLAM faces several challenges, including:

- **Data Association**: It can be difficult to correctly match incoming sensor data to previously mapped locations, particularly in visually similar or changing environments.
- **Computational Complexity**: Processing the sensor data in real-time while maintaining accuracy can be computationally expensive, especially in large, complex spaces.

Two widely used SLAM techniques in ROS include:

1. **Gmapping**: A 2D SLAM algorithm based on particle filters, which is well-suited for LIDAR data and provides probabilistic mapping and localization.
2. **RTAB-Map**: A 3D SLAM method using RGB-D cameras for appearance-based mapping. It is efficient in terms of memory usage and can handle larger environments with visual landmarks.

Both techniques integrate well with ROS and are commonly used in robotic systems.

SLO 3.2 Demonstrating the SLAM task in Gazebo environment

Figure 3-0-1 Top view of environment with RTAB map overlay

Initially the map was produced using the SLAM toolbox, but due to the generation of a skewed map, there was misalignment, due to this RTAB was used to produce a more accurate map. However, RTAB posed a couple of issues due to being a large program to run was very slow and would often time out. Due to this it was created on Andrew's computer using dual boot and then was shared with myself and the rest of the group. There was also lack of knowledge using RTAB and as such had to familiarize myself with how to navigate and run using it.

One slight issue encountered toward the end as the node was functioning was the map and image did not align and varied in its inaccuracies on each side of the image. As there was no easily implementable function to scale sections of the background image, the solution was to crop the image to fit the map better. This works as intended allowing the image to better overlay.

SLO 3.3 Analyse computational aspects in simulations

Computation time for the simulation

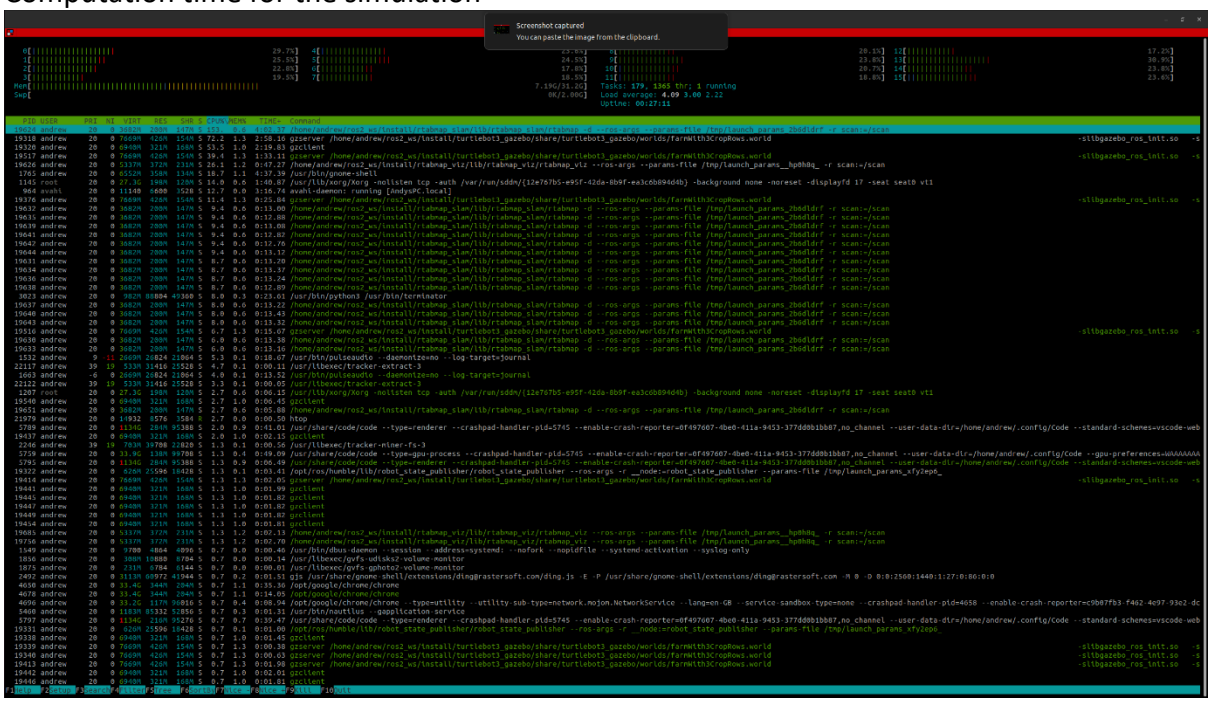


Figure 3-0-2 Computation time from terminal

SLO 3.4 Provide a short description of the Robot Path Planning task

Path planning refers to the process of determining an efficient and collision-free path for a robot to move from its starting position to a target destination. It is essential for autonomous navigation and involves finding a balance between minimizing travel distance, time, and avoiding obstacles within the environment.

- **A***: A heuristic-based search algorithm that finds the shortest path by considering both travelled distance and the estimated cost to the goal (Hart et al., 1968).
- **RRT (Rapidly-exploring Random Tree)**: A randomized algorithm that efficiently explores large spaces by building a tree of possible paths (LaValle, 1998).
- **Dijkstra's Algorithm**: A graph-based algorithm that finds the shortest path by uniformly expanding from the start node (Dijkstra, 1959).

Challenges in path planning include navigating dynamic environments where obstacles or goals may move, ensuring real-time computation in complex or cluttered spaces, and balancing between path optimality and computational efficiency. Additional challenges include safety considerations and resource constraints, such as energy or time.

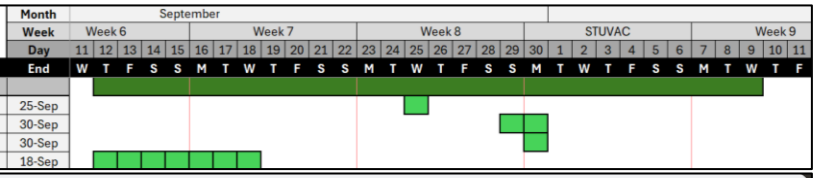
SLO 3.5 Demonstrating the path planning task in the Gazebo environment

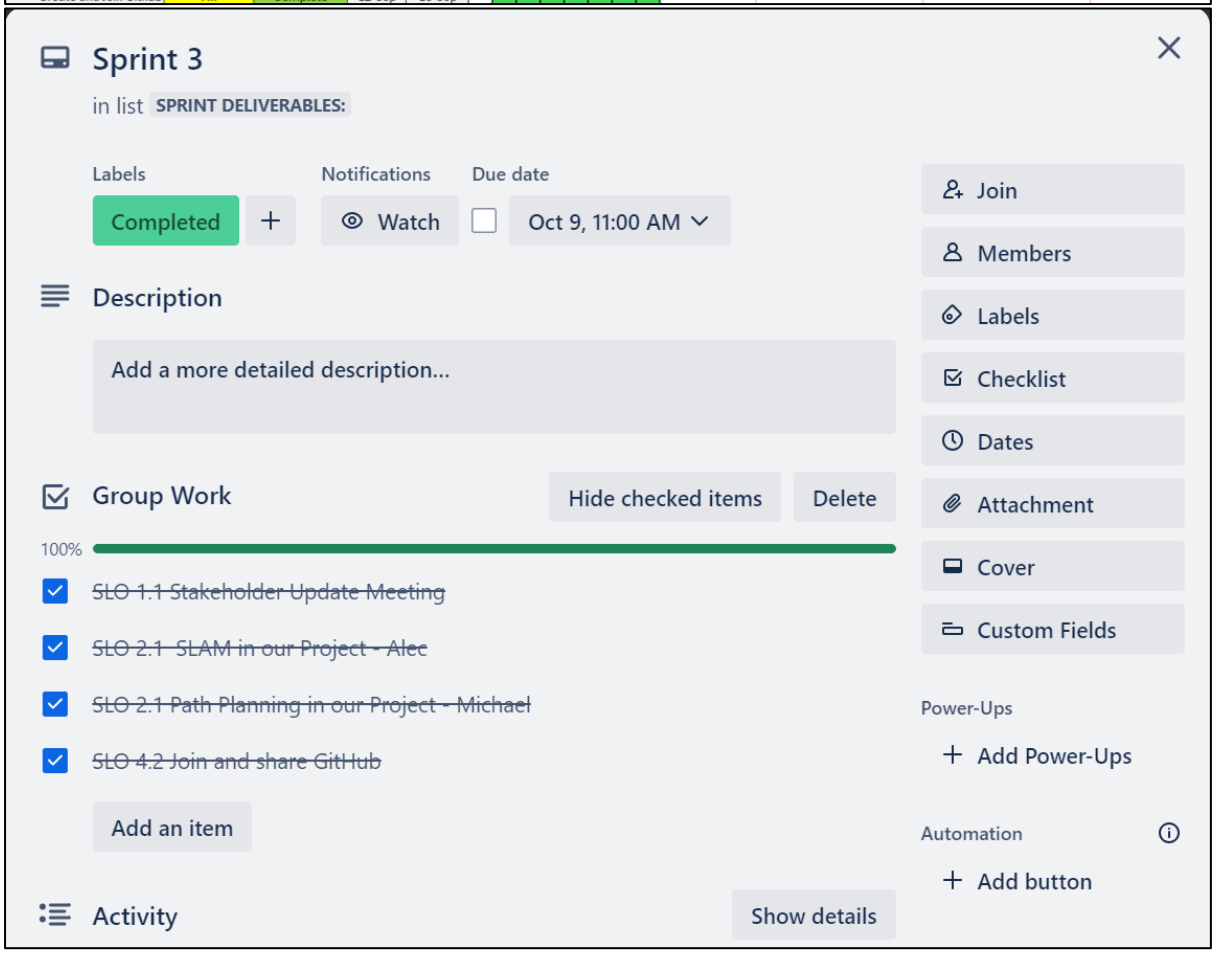
Not too many challenges, it was gaining an understanding of the parameters and their function within the path planning. A lot of the time was spent experimenting with these parameters and figuring out what would need increasing or decreasing to better optimize the pathing.

SLO 4

SLO 4.1 Time management

Activity	Assigned To	Status	Start	End
Sprint 3				
Stakeholder Meeting	All	Complete	25-Sep	25-Sep
SLAM in our Project	Alec	Complete	29-Sep	30-Sep
Path Planning in our Project	Michael	Complete	30-Sep	30-Sep
Create and Join Github	All	Complete	12-Sep	18-Sep





SLO 4.2 Create a GitHub repository

A link to the Crop Guard team repository can be found here.

SLO 4.3 Professionally formatted portfolio and Doxygen

Cylinder Detection

This node takes on the task of detecting a cylinder object with a diameter of 30cm in the Crop Guard simulated farm environment, this code will be the precurser to our obstacle detection functionality

Main Page | Classes | Files | Search

Cylinder Detection System

Introduction

This documentation provides an overview of the **Cylinder** Detection Node, designed to detect cylindrical objects using laser scan data in ROS 2. The system processes scan data to find cylinder-shaped objects in the environment and visualizes them using RViz markers.

Node Setup

The system consists of the `Cylinder` node, which subscribes to laser scan and odometry topics to detect and track cylinders.

Cylinder Node

The `Cylinder` node subscribes to laser scan data (`/scan`) and odometry data (`/odom`). It processes the laser scan data to identify segments and detects cylinders by approximating circles using three points from a segment. Detected cylinders are visualized in RViz via markers.

Cylinder Detection Algorithm

The detection process follows these steps:

1. **Segment Identification**: Laser scan points are segmented based on proximity.
2. **Cylinder Detection**: Segments are analyzed by checking the curvature formed by three points, estimating a cylinder based on radius matching.
3. **Visualization**: Once a cylinder is detected, it is visualized using `visualization_msgs::Marker` in RViz.

Cylinder Detection Behaviour

1. **Laser Scan Subscription**: The node receives laser scan data and segments points based on proximity.
2. **Odometry Update**: The odometry data is used to transform local laser scan points into global coordinates.
3. **Cylinder Detection**: The node checks for circular patterns in the segments and confirms cylinder detection by matching the radius.
4. **Visualization**: Detected cylinders are visualized with cylinder markers in RViz.

How to Run

- Open Gazebo Environment:
```
export TURTLEBOT3_MODEL=waffle_pi
ros2 launch turtlebot3_gazebo farmwithCylinder.launch.py
```
- Open RViz with Pre-loaded Map:
```
export TURTLEBOT3_MODEL=waffle_pi
ros2 launch turtlebot3_navigation2 navigation2.launch.py map:=$HOME/ros2_ws/RTAB_MAPS/farmWith3CropRows/map_1728280899.yaml
```
- Launch the Node:
```
ros2 run cylinder cylinder_node
```
- Visualize Cylinders: Set RViz to visualize detected cylinders published on the /visualization_marker topic.
- Teleop around the environment:
```
export TURTLEBOT3_MODEL=waffle_pi
ros2 run turtlebot3_teleop teleop_keyboard
```

What to Expect

Running this code you can expect that when you teleop around the environment with a cylinder object previously unmapped. The code will detect the cylinder object and place a marker on the map which corresponds to it's global coordinate in the map.

SPRINT 4

SLO 1

SLO 1.1 Stakeholder engagement on project completion

Table 4.1 - Meeting Minutes with subject stakeholder

Meeting Name:	Stakeholder Handover Meeting	Meeting Date:	29/10/2024
Attendees:	Andrew, Alec, Anika, Dylan, Michael, Jaydie, Hugh Byun (Client)		

Agenda:

- Team to demonstrate final project deliverable.
- Client to ask team questions about the approaches taken in the project and to provide some brief feedback to the team.

Summary of Discussion Notes:

- Project Simulation:
  - Team demonstrated the project code being run through the GUI.
  - Team explained the procedure of setting up the program as well as the logic it follows as the progresses.
  - Client asked the team questions about methods used for mapping, localisation, path planning.
- Additional Considerations:
  - Client asked the team questions about additional considerations for the project code such as:
    - Dynamic path planning
    - Obstacle detection and avoidance
    - Additional sensors
    - Other localisation and mapping methods
  - Client asked the team questions about realising the project including:
    - Logistics of implementing the project
    - Flexibility of the project for different environments
    - Financial cost of implementing the project
    - Sustainability of the project

Action items:

- None

SLO 2

SLO 2.1 Develop an evaluation criteria

Table 4.2 - Technical Aspects of the Project

Criteria	Significance	Demonstration
Robot Mapping	The robot needs to traverse its environment to create a map that it can later use for, localisation, path planning and navigation.	The main project used the RTAB-Map package to develop a high-quality map for use in the project. However, Azure systems struggle to run this package and use the slam_toolbox instead.
Robot Localisation	The robot needs to be able to calculate its location within the predetermined map using the live sensor data.	The project uses the Rviz 2D Pose Estimate package to localise the robot within the environment using a manually input estimate.

Table 4.3 - Client Assessment of the Project

Criteria	Achieved	Unachieved
Does the system effectively identify healthy and unhealthy crops?	The system uses a custom-made colour detection function using OpenCV to assess the health of crops. It can successfully determine between a green (healthy) plant and a yellow (unhealthy) plant.	The system does not yet take into consideration the soil composition around the plant when making its decision about plant health.
How often does the system make mistakes in its assessment of crop health?	During testing, the system seemed to be accurate at assessing plant health on colour 100% of the time.	Although the testing of the system proved promising results, it has not been rigorously tested to find use cases where it may potentially turn up incorrect assessments.
Does the project provide users with helpful information?	The system places an indicator on the map when and where an unhealthy plant is detected as well as output a warning to the terminal.	The system could not be made to produce a .txt file report at the end of each execution as initially intended. The GUI also does not yet display the live camera feed.
Is the final deliverable formatted and documented to professional quality?	The project code files are all contained within a single package and are neatly formatted in a GitHub repository.	The project does not have detailed Doxygen documentation describing its functionality. It relies on internal code comments for brief explanation.
Is the final deliverable's running performance acceptable?	The project code runs very smoothly and without glitching on dual booted computers.	The project environment had to be severely simplified and reduced to work using Azure VMs. Even in this case, it can still be quite glitchy and often slow.
Is the project financially viable?	The system only requires the use and support of a single robot, making it much cheaper to operate than a team of farm hands.	The development of this project and its custom robot is not fully completed. Thus, a full financial burden cannot be determined.
Is the project sustainable?	The project is sustainable due to the reduced carbon footprint associated with replacing heavy farming machinery with an electric robot.	The development of this project is not fully completed. Thus, a full sustainability impact analysis cannot be calculated.

SLO 3

SLO 3.1 Project evaluation and testing: Overall

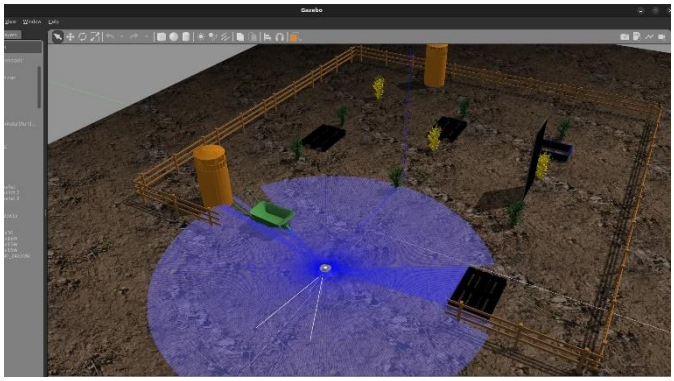


Figure 4-1 *- Final project Gazebo Environment*

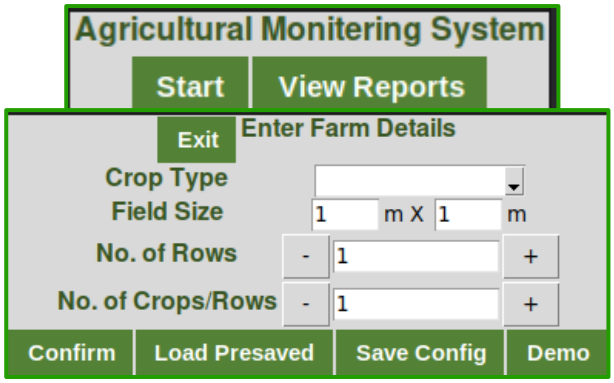


Figure 4-2 *– GUI of project*

The achievements of the Crop Guard project are substantial, showcasing both the project's breadth and successful outcomes. The final deliverable comprises a meticulously designed Gazebo simulation that realistically mirrors a farm setting, complete with crop rows, farm machinery, and various obstacles, as shown in Figure 30. The system operates through a dedicated GUI, which initializes the ROS node and launches both Rviz and Gazebo, as illustrated in Figure 4.1. Following predefined path instructions, the TurtleBot3 navigates the environment, scanning and evaluating plant health based on colour recognition. Instant feedback on each crop's health status is then displayed in the terminal, with any unhealthy crops distinctly marked on the Rviz map, allowing users to swiftly identify affected plants, as shown in figure 4.2.

While the project achieved considerable success, certain planned features were not completed as initially envisioned. Notably, the integration of a simulated nitrate sensor for deeper insight into plant health was deprioritized as project goals evolved. This feature would have involved a script executed at initialization, partitioning the Gazebo environment into cells with assigned nitrate levels, which the TurtleBot3 would then read and assess. Additionally, the team had intended to incorporate a live camera feed within the GUI to provide real-time visual feedback. This feature would have required integrating ROS subscribers and publishers within the GUI code, allowing users to access a live camera view directly. However, due to adjusted priorities and time constraints, this function was ultimately omitted from the final deliverable.

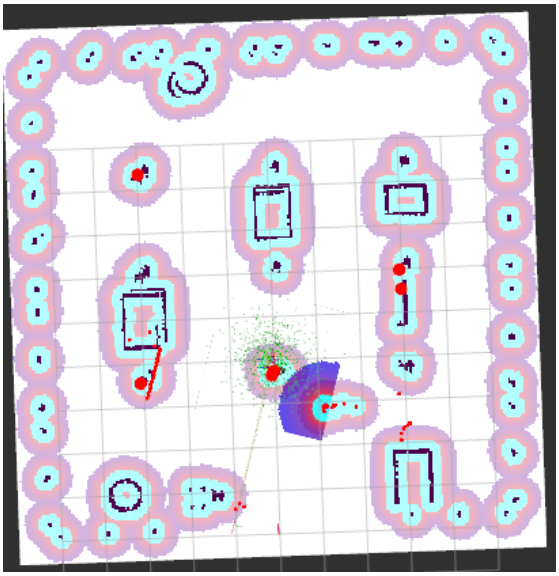


Figure 4-3 *- Rviz map of unhealthy crop marks*

SLO 3.2 Project evaluation and testing: Mapping

Robotic mapping is integral to the effective operation of the Crop Guard system, enabling the TurtleBot3 to survey crop fields and identify accessible and restricted areas. Through this process, the system generates an occupancy grid map based on scanned data, helping prevent collisions with the environment, crops, or other obstacles. Additionally, the generated map is instrumental in creating path plans and optimizing routes between waypoints. Mapping also supports localization by providing reference landmarks to calibrate the robot's pose and position accurately.

Several challenges emerged during mapping implementation. First, map generation proved time-consuming, as creating each new map required manually guiding the robot throughout the environment, often with multiple passes to capture sufficient detail. Another issue arose if the robot collided with an object or encountered uneven ground; these disruptions would cause laser scans to detect false anomalies, resulting in mapping errors and requiring a complete map regeneration to correct.

In ROS2, the slam_toolbox is a popular choice for SLAM and mapping, offering a reliable, straightforward platform that integrates seamlessly with Gazebo and Rviz, utilizing loop closure detection. However, it is limited to 2D environments. For more extensive 3D mapping, the RTAB-Map package is a stronger option, producing high-quality maps by leveraging visual and depth data, though it demands significantly higher computational resources to operate effectively.

SLO 3.3 Project evaluation and testing: Localisation

Localization played a critical role in the Crop Guard project. The approach involved aligning live laser scan data from the TurtleBot3 with pre-existing map data generated from prior scans. By using an initial pose estimate, the system could locate and orient the robot accurately on the map.

A primary challenge encountered during localization arose from the repetitive nature of the farm environment. The similar layout of rows and columns led to multiple positions having indistinguishable laser scan profiles, confusing the robot during localization. This issue was mitigated by adding distinctive objects to the environment, ensuring that each position had a unique scan pattern. Another issue emerged when the TurtleBot3's laser scan data became unreliable while moving. Bumps in the terrain occasionally caused the robot to misidentify its location on the map, impacting path planning and navigation. To address this, the robot's top speed was reduced, and the ground surface was leveled as much as possible to minimize scan errors.

The Crop Guard project used the Rviz 2D Pose Estimate package, a practical tool that allows users to manually provide an initial position and orientation to aid robot localization. However, it lacks continuous localization capabilities. An alternative package, Robot_Localization, offers continuous, multi-sensor localization in both 2D and 3D, integrating IMU and wheel encoder data for enhanced accuracy. However, its complexity and reliance on high-quality sensor data make it more challenging to configure.

SLO 3.4 Project evaluation: Client's assessment of completeness of the project

During the stakeholder handover meeting, we presented our project to our client (Hugh), showing of the systems completeness and functionality. We further discussed aspects to be implemented given more time before real world application. For more information on this look to Sprint 4, SLO 1.1.

SLO 4

SLO 4.1 Time management

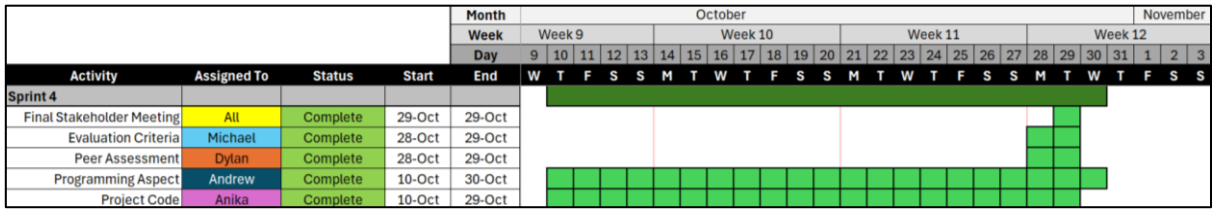


Figure 4.4 – Gannt Chart with Group Tasks for Sprint 4

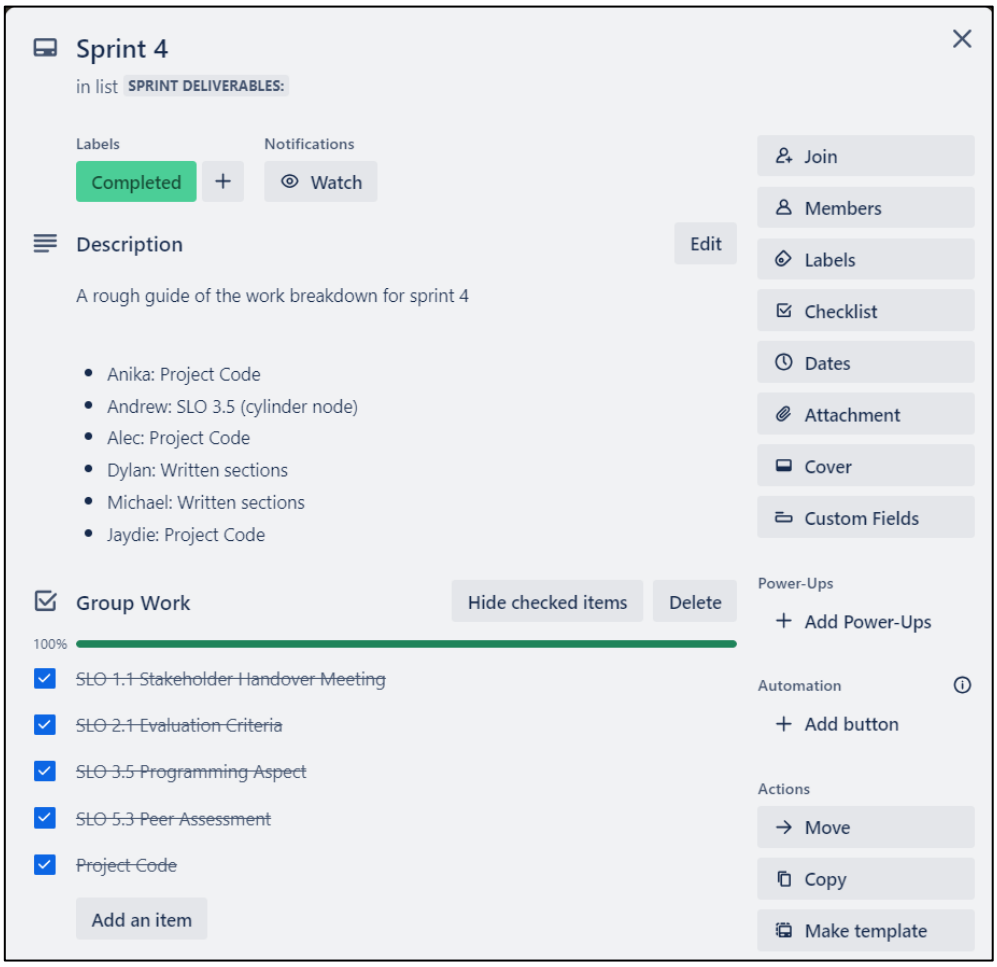


Figure 4.5 – Completed Trello Card for Sprint 4 with all Group Tasks and Division of Tasks

SLO 5

SLO 5.1 Critical reflection on the subject learning

Each sprint required the completion of numerous Subject Learning Objectives (SLOs) to demonstrate proficiency across multiple areas. As the team project manager, I took the initiative to thoroughly review the marking criteria at the start of each sprint, extracting detailed information and interpreting the requirements of each SLO. I then compiled these insights along with personal notes into a shared document accessible to the team, outlining specific task expectations. This transparency facilitated the division of group work into equitable sections, enabling team members to work independently or collaboratively as needed to fulfill each SLO.

One recurring challenge was interpreting the sprint marking criteria, as the descriptions of certain SLOs were often vague or ambiguous, creating confusion within the team. To address these issues, I reached out to fellow students for clarification on unclear aspects, consulted with my class tutor, and posted questions on Microsoft Teams to receive guidance directly from subject staff.

Another issue that arose of the course of the project was technical difficulties. This included computational power lacking when it came to simulating the environment and a few of the tasks put to us. As well as issues with Linux itself, the OS corrupting prior to the lab demonstration day of sprint 3 and having to reinstall and setup the entire Linux environment alongside Rviz, Gazebo and the other tools and files used for this project and workshops held.

SLO 5.2 Statement of what you will you do differently

If given the opportunity to retake this subject and project, there are several areas I would approach differently to optimize both project management and workflow. Primarily, I would prioritize securing a more robust Ubuntu installation on a dual-boot system to ensure sufficient computational power for running intensive programs. Alongside this, I would keep an Azure virtual machine (VM) as a secondary setup to maintain continuity in case issues arise with the dual-boot system. The lack of computational power in the Azure VM was a recurring obstacle, often slowing down workflow and extending computation times, which impacted project efficiency. A more powerful local setup would enable smoother processing, allowing me to run complex simulations without frequent delays or system crashes.

Additionally, I would improve the organization and structuring of both my own time and the team's work schedule. Factors such as task misinterpretations, unexpected technical issues, delays, and occasional team member absences often led to compressed timelines, requiring us to rush through sections of the project. To counter this, I would aim to thoroughly understand and complete lab assignments, gaining the technical foundation needed to apply those skills more effectively in the project. Furthermore, I would emphasize the importance of meeting project deadlines within the team to prevent bottlenecks. I would also prioritize early communication with my class tutor to clarify any ambiguities in sprint requirements or marking criteria. Proactively addressing these challenges would establish a clearer, more structured workflow, enabling the team to focus more on quality deliverables and reduce time spent deciphering project guidelines.

SLO 5.3 Peer assessment – Group Assessed "Book Worms"

Assessment to Book Worms

The Book Worms group is working on an autonomous librarian project to develop a robot that assists librarians by saving time on tasks such as locating books, guiding patrons, and identifying book damage. The core problem being addressed is optimizing librarians' time through automation while enabling safe and efficient navigation within a library environment.

To tackle this, Book Worms has incorporated SLAM (Simultaneous Localization and Mapping) to create a 2D map of the library. SLAM, along with robot localization and path planning, allows the robot to navigate the library efficiently. Additional functionality, such as a precise book-location system, is required for the robot to autonomously guide users to specific bookshelves, and this feature is still under development.

Key challenges include localization issues due to the robot's movement speed at the sensor range's edge, leading to position shifts, and the limited view angle of the RGB camera, which reduces data quality. The robot's stopping mechanism, which controls orientation and distance from shelves, also remains a work in progress. Although navigation and pathing are implemented, the book-location functionality is incomplete, making this an ongoing project with substantial refinement needed.

Assessment from Book Worms

CropGuard is an autonomous robotic analysis system, which enables the collection of various key data points from crops in large agricultural settings. The aim is to reduce the total workload on farmers and farmhands, by significantly reducing outsourcing requirements, and enable the analysis of data through generated reports from the robot. The use of SLAM is done primarily through the slam_toolbox's AMCL localisation and navigation. Mapping tasks were performed using RTABMap. Their use of additional functionalities, including extra sensors for nitrate and moisture content, with camera image recognition to determine plant health, are used in the report generation that the robot automatically completes. Given the complexity of the task, challenges that CropGuard faced, were poor localisation performance through AMCL, given that the map contained little to gather data from. Path planning to effectively work with camera scanning, and changing certain configurations to further optimize navigation speed.

SLO 5.4 Concluding statement

The Crop Guard project successfully created a realistic simulated environment, utilizing a TurtleBot3 to autonomously navigate the field and assess crop health. The system identified unhealthy crops by analyzing both their color and soil composition, and once detected, flagged the location by marking the area on the pre-scanned map and notifying the user. This project incorporated various robotic control methods, physical simulation hardware, and C++ programming, allowing for a comprehensive approach to autonomous monitoring.

Throughout this subject, I developed a foundational understanding of essential robotics principles, including localization, mapping, path planning, and SLAM. These skills are integral to autonomous robot functionality, each contributing to control logic development. Additionally, I became proficient in the use of Gazebo and Rviz, gaining confidence in setting up simulated environments and observing the impact of ROS topics on the system. I also gained awareness of valuable resources available for robotics projects, particularly those on platforms like GitHub and the ROS2 website, which support efficient problem-solving and skill enhancement.

The knowledge gained through this project will be highly advantageous for my future studies and career in mechatronics engineering. As I work on similar robotics projects, I expect to encounter challenges like those addressed in this subject. Equipped with the skills and insights gained here, I am optimistic about tackling future robotics problems more effectively. Additionally, with the ROS2 support resources I used during this project, I feel prepared to navigate challenges that may arise in future projects with greater confidence and direction.

References

Dijkstra, E. W. (1959). *A note on two problems in connexion with graphs*. Numerische Mathematik.

Grains Research and Development Corporation (GRDC). (2022). *Identification and Management of Field Crop Diseases in Victoria*. https://agriculture.vic.gov.au/__data/assets/pdf_file/0012/901101/Field-Crop-Diseases-ID-manual-2022small.pdf

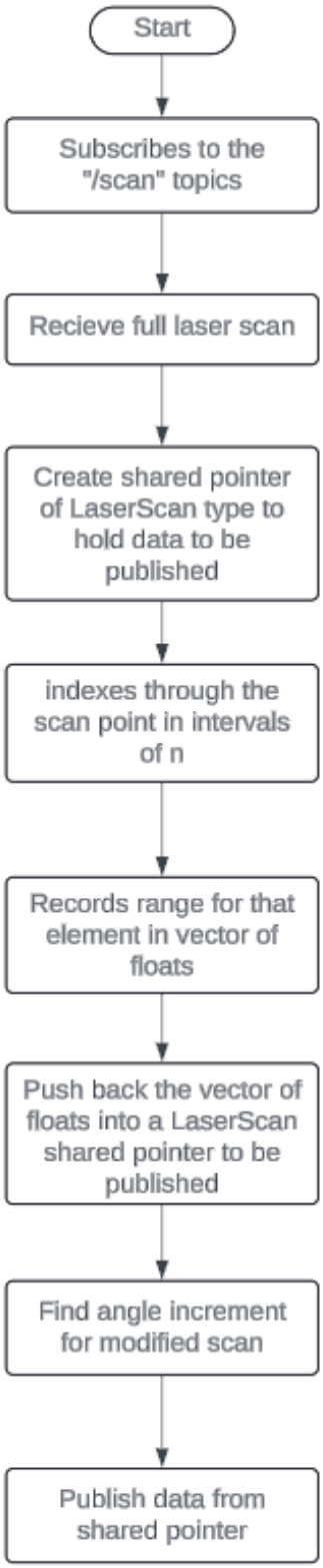
Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). *A formal basis for the heuristic determination of minimum cost paths*. IEEE Transactions on Systems Science and Cybernetics.

LaValle, S. M. (1998). *Rapidly-exploring random trees: A new tool for path planning*. Technical Report TR 98-11, Computer Science Department, Iowa State University.

Whitlock, A. (2013, Aug 29). How to Determine the Ultimate Row Spacing. GRDC. https://grdc.com.au/resources-and-publications/grdc-update-papers/tab-content/grdc-update-papers/2013/08/how-to-determine-the-ultimate-row-spacing#:~:text=Some%20farmers%20can%20retain%20stubble,decline%20as%20row%20spacing%20increases.

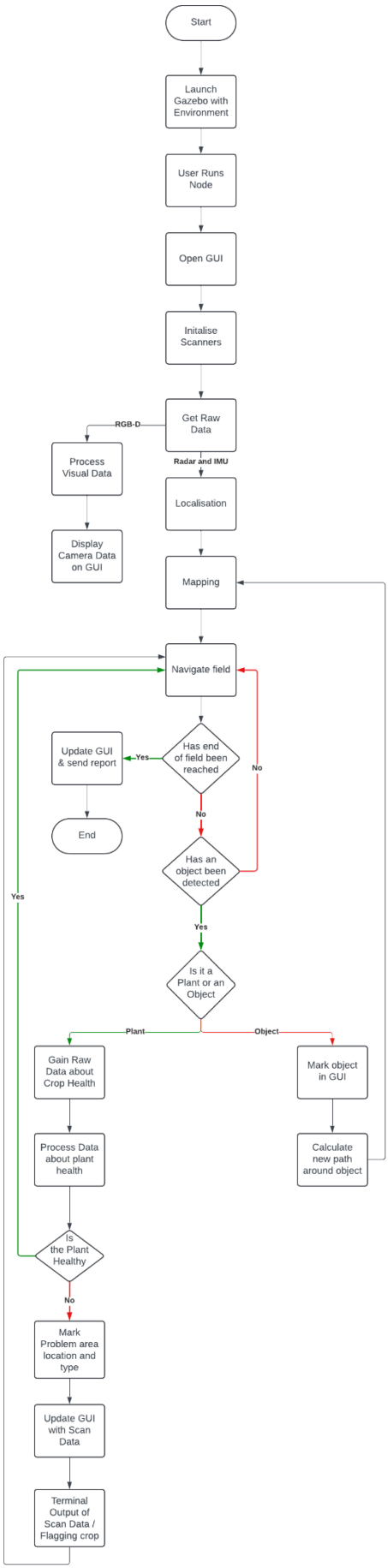
Appendix

Appendix A: Subscriber & Publisher Flowchart



Appendix B: Risk and Mitigation Strategy Matrix

RISK NUMBER	PRE-MITIGATION				MITIGATION	POST-MITIGATION			
	RISK	RISK SEVERITY	RISK LIKELIHOOD	RISK LEVEL		RISK SEVERITY	RISK LIKELIHOOD	RISK LEVEL	ACCEPTABLE TO PROCEED?
1	Injury of person due to interaction with robot	INTOLERABLE	POSSIBLE	EXTREME	Implementation of emergency stop/overide with force sensors	ACCEPTABLE	IMPROBABLE	MEDIUM	YES
2	Malfuction of hardware	INTOLERABLE	POSSIBLE	HIGH	checking over equipment before use, regular maintenance checks	ACCEPTABLE	IMPROBABLE	LOW	YES
3	Injury from poor posture	UNDESIRABLE	POSSIBLE	MEDIUM	consistently taking breaks, ensure ergonomic workspace	ACCEPTABLE	IMPROBABLE	LOW	YES
4	Sensory pain - eye strain/headache	UNDESIRABLE	POSSIBLE	MEDIUM	sharing workload/taking breaks	ACCEPTABLE	IMPROBABLE	LOW	YES
5	Exposure to extreme/harsh weather	TOLERABLE	PROBABLE	LOW	Tracking weather and not operating robot if weather too harsh	ACCEPTABLE	IMPROBABLE	LOW	YES
6	Exposure to/Interaction with wildlife	TOLERABLE	PROBABLE	LOW	implementation of a system to detere wildlife interaction with robot	TOLERABLE	IMPROBABLE	LOW	YES
7	Crop misidentification (both colour and type)	INTOLERABLE	POSSIBLE	MEDIUM	provide a large bank of reference for identification model	TOLERABLE	IMPROBABLE	LOW	YES
8	Localization system failure	UNDESIRABLE	POSSIBLE	MEDIUM	incorporation of secondary localization method implement fallback behaviour to a known safe location until localization is restored	ACCEPTABLE	IMPROBABLE	LOW	YES
9	Sensor Failure	INTOLERABLE	POSSIBLE	MEDIUM	Regular testing of sensors and setting up alets for irregular sensor behavior	ACCEPTABLE	IMPROBABLE	LOW	YES
10	Software error - eg failure to compile data	INTOLERABLE	POSSIBLE	MEDIUM	rigorus testing of software processes	ACCEPTABLE	IMPROBABLE	LOW	YES

Appendix C: Overall Project Psuedo Code and High-Level Flow chart

```
START initialisation
FOR X time
        START scanning
        START navigation
END FOR

FUNCTION NAVIGATION
IF end of field reached
        END program
ELSE
        IF object detected
                IF object is a plant
                        COLLECT crop health data
                PROCESS crop health data
                IF plant is unhealthy
                        MARK problem area on GUI
                        START Terminal Output
                ENDIF
        ELSE
                MARK object on GUI
                START Terminal Output
                CALCULATE new path around object
MOVE along new path
                ENDIF
        ELSE
                MOVE forward
                ENDIF
        ENDIF
END FUNCTION

FUNCTION INITIALIZE
        LAUNCH Gazebo with environment
        WAIT for user to run node
        ACCESS inputted map
        CALCULATE optimum path
        OPEN GUI
END FUNCTION

FUNCTION SCANNING
        GET data from RGB-D Camera
        PROCESS visual data
        DISPLAY visual data on GUI
        GET data from Radar
        START Localisation
END FUNCTION

FUNCTION LOCALISATION
        RETRIEVE RGB-D and radar data
        COPY Initial picture
        COPY current picture
        COMPARE pictures
        ORIENTATE robot
        MOVE robot
        REPEAT till pictures align
END FUNCTION

FUNCTION TERMINAL_OUTPUT
        UPDATE flag crop in GUI
        OUTPUT scan data and flag in terminal
END FUNCTION
```